

U1L3 Guided Notes: Exploring Data Types

Learning Objective: I can use and describe Python's data types (strings, integers, floats, and booleans) and explain how they are used in simple programs.

Language Objective: I can talk and write about data types by describing what kind of information each one stores, using complete sentences and Python examples (for example, "An integer stores whole numbers like 4 or 9.").

What Are Data Types?

Data types define the kind of _____ a variable stores.

Data Type	Description	Example
String		"Hello"
Integer		
Float		
Boolean		

Strings (str)

- A **string** is text inside quotes: "Hi" or 'Python'.
- You can **join** strings using + or **commas in print()**.

Concatenation (+) example:

```
greeting = "Hi"
name = "Alex"
print(greeting + " " + name)
```

Commas Example (no conversion needed):

```
print("Hello", name)
```

Integers and Floats

- **Integers** are whole numbers (like 7 or -4).
- **Floats** are numbers with decimals (like 3.14).
- You can do math with both!
 - Integer + Integer → Integer
 - Float + Float → Float
 - Integer + Float → Float

```
num_apples = 5
price_per_apple = 0.75
total_cost = num_apples * price_per_apple
print("Total cost:", total_cost)
```

Check Yourself:

- What is the data type of `num_apples`? _____
- What is the data type of `total_cost`? _____

Booleans (**bool**)

- A **Boolean** represents a value that is either **True** or **False**.
- It's used to show **yes/no**, **on/off**, or **is/is not** conditions in your code.
- Only these two words are Booleans — they must start with **capital letters**:
 -  **True**
 -  **true** (not valid in Python)

```
is_light_on = True
print("Light on?", is_light_on)
```

Common Uses:

- To represent simple states (on/off, yes/no).
- To check if something is happening or not.
- To make decisions later in code using conditions (**if/else**).

Be Careful:

If you put quotes around `True` or `False`, Python sees it as a **string**, not a Boolean.

```
is_logged_on = "true"      # ❌ this is a string
print(type(is_logged_on))  # Output: <class 'str'>

is_logged_on = True        # ✅ this is a Boolean
print(type(is_logged_on))  # Output: <class 'bool'>
```

Try It:

1. Create a variable called `is_student` and set it to either `True` or `False`.
2. Print it with a label (e.g., `print("Student status:", is_student)`).
3. Then test what happens if you write `"True"` instead of `True`.
4. Write down the difference between the two in your notes.

Type Conversion

When you mix data types (for example, trying to join text with numbers), you might get an error.

Example Error:

```
paint_colors = 12
print("You bought " + paint_colors + " colors of paint.")
```

Result:

```
TypeError: can only concatenate str (not "int") to str
```

Fixed Version:

```
print("You bought " + str(paint_colors) + " colors of paint.")
```

Why?

Python can only concatenate strings, so you must convert numbers using `str()` before combining them with text.

Using `type()`

Use the `type()` function to check a variable's data type.

```
name = "Alex"
age = 16
print(type(name))
print(type(age))
```

Write your own mini test below:

1. Create a variable.
2. Use `print(type(...))` to check it.
3. Write what Python tells you.

Debugging Practice

Example 1 – Concatenation Error

```
score = 25
print("Your score is " + score)
```

What error do you think will happen? _____

Fix it below:

Example 2 – Boolean Fix

```
is_logged_on = "true"
print("Login status:", is_logged_on)
print(type(is_logged_on))
```

What’s wrong with the code? _____

Rewrite it so there are no errors:

Quick Reference

Function	What It Does	Example
type()	Checks a variable’s data type	type(age)
str()	Converts to string	str(10)
int()	Converts to integer	int("7")
float()	Converts to float	float("2.5")

Add any additional notes below: